



BRENT OZAR
UNLIMITED®

Cloud Storage Performance

Deck 3 p1

This affects both IaaS and PaaS.

No matter how you deploy databases, this is relevant.

You can fix it by throwing money at hardware,
but you have to know which dial to turn.

There isn't just one hardware dial.



Deck 3 p2

Agenda

Where hardware shows up in query plans

Hardware evolution from 2000s to 2020s to cloud

Why cloud storage performance is catastrophic

Why memory isn't the answer it was on-premises

Dealing with 2000s-quality hardware in the cloud

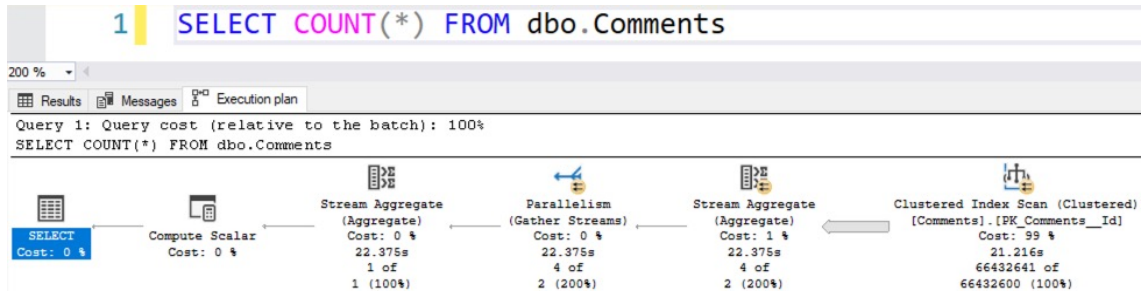
- Groups of drives to pool performance
- Choosing your VM type to maximize throughput

Deck 3 p3



How hardware
shows up in plans

Execution plans stop at indexes.

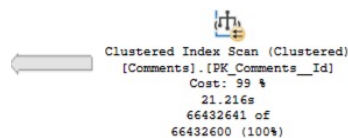


But there's more to the right.

Deck 3 p5



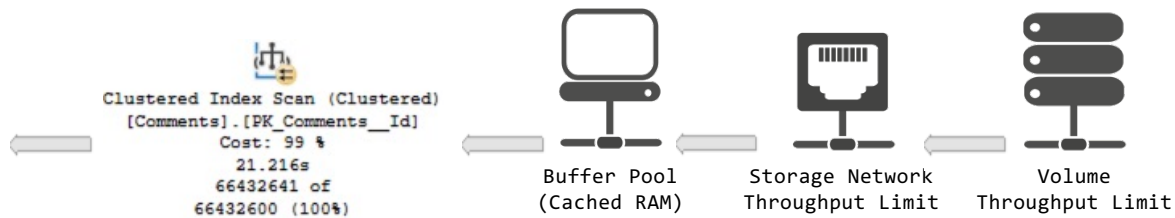
How does this part really work?



Deck 3 p6



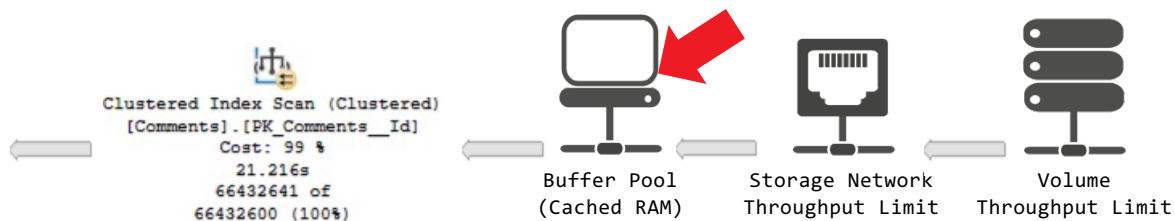
The plan really keeps going.



Deck 3 p7



If you can cache all data in RAM



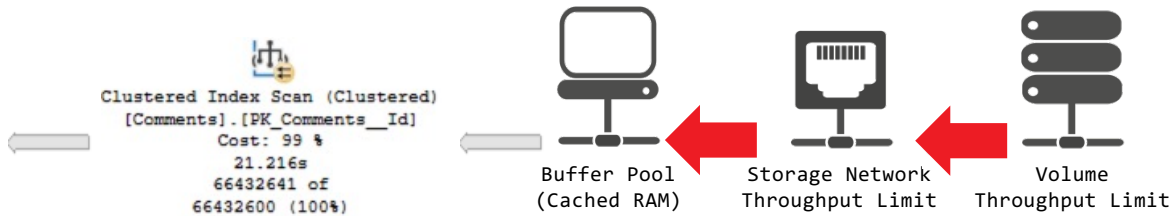
Then storage throughput only matters:

- On startup
- For backups
- For inserts/updates/deletes (to the log file)

Deck 3 p8



If you have to read data from disk



Then the smallest red arrow dictates read query performance.

Load testing each arrow was hard.

Deck 3 p9



This was terrible in the 2000s.

We simply couldn't cache enough data.

We used magnetic hard drives, so:

- Drives only did 150 operations per second (IOPs)
- We pooled drives together in RAID arrays not just for safety, but for speed

1 TB was called a Very Large Database (VLDB)

Deck 3 p10



But in the 2010s...

Memory costs dropped from \$100/GB to \$10/GB:
cache 10x more, cheaply. 128GB = \$1,280.

Fusion-io and Intel popularized solid state drives:
IOPs per drive went from 150 to 150,000+

SAN vendors like Pure built fast shared storage:
a shared pool of flash had nearly unlimited throughput

Deck 3 p11



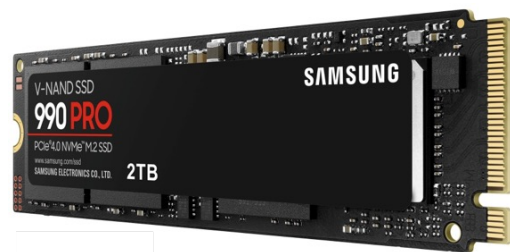
Amazing time to be alive

Samsung 990 Pro for laptops

Up to about 1.5 million IOPs
(that's 10,000 hard drives)

Up to 7,450 MB/sec
(that's 6 10Gb iSCSI cables)

1TB \$200, 2TB \$300



Deck 3 p12



Better than flying cars

	2000s	2010s	2020s
Cost for 128GB RAM	\$12,800	\$1,280	\$500
Typical drive IOPs	150	150,000	1,500,000*
Typical drive throughput	100 MB/sec	1,000 MB/sec	7,000 MB/sec*

* We're only in 2022, and consumer-grade drives are already available that meet these specs.

Deck 3 p13

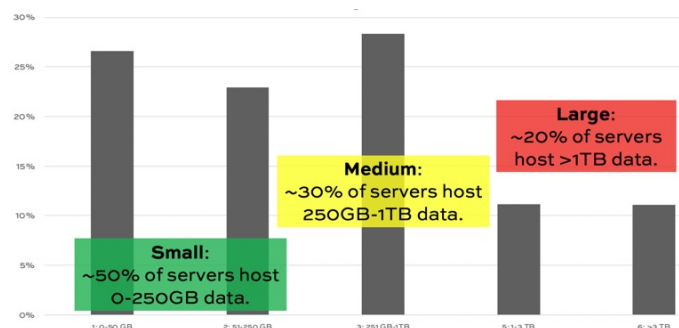


So we didn't mind growing data.

1 in 5 servers have
1TB or more.

It's not very large now.

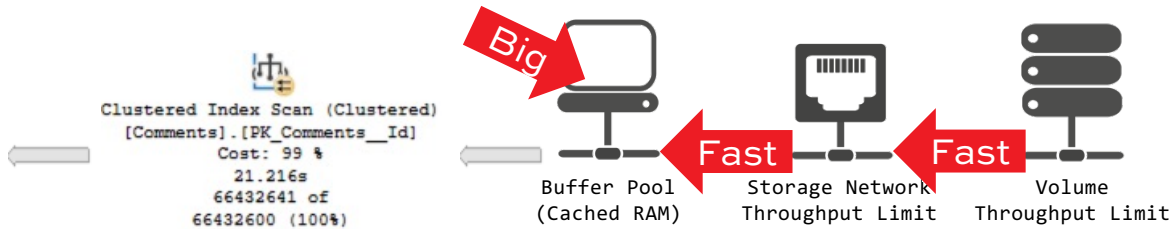
But who cares?
Hardware got so good,
we could deal with it.



Deck 3 p14



In the 2020s



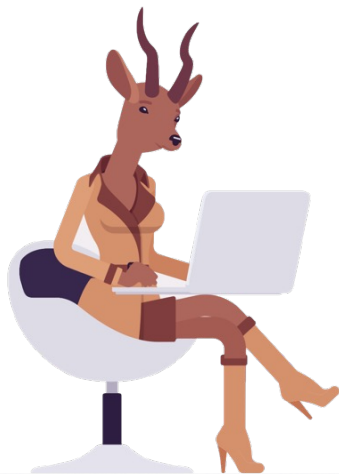
Memory grew so large,
and storage throughput grew so fast,
that many servers just didn't have to worry about it.

Blocking, CPU, parallelism were bigger issues.

Deck 3 p15



Alex the Architect's hardware



Alex just manages one application & db

The app is growing fast & furious,
users constantly want more features

Can't hire enough good developers

Tends to overprovision hardware,
throwing hardware at problems,
so tends to have high quality hardware

Uses 2020s-grade hardware

Deck 3 p16



Billie the Small Biz Bird's hardware



Billie's business sees IT as an expense

The business wants to spend the bare minimum possible to keep apps online

The business will tolerate some outages to save money.

Billie uses a couple/few VM hosts backed by inexpensive shared storage.

Deck 3 p17



Elliot's enterprise hardware

Elliot's team uses large amounts of hardware from enterprise vendors

A mix of virtualization and bare metal

Refreshes hardware on a scheduled basis, like every 3-5 years

Tries to balance performance and cost

Avoids bleeding edge hardware, so today's hardware is 2010s-grade performance

Deck 3 p18



Then came the cloud.

Uh oh

	2000s	2010s	2020s	2020s Cloud
Cost for 128GB RAM	\$12,800	\$1,280	\$500	\$11,700/yr* (E16bds_v5)
Typical drive IOPs	150	150,000	1,500,000	5,000 (Azure P30 1TB)
Typical drive throughput	100 MB/sec	1,000 MB/sec	7,000 MB/sec	200 MB/sec (Azure P30 1TB)

*This is the cost premium per year for an E32 over an E16, which takes you from 128GB to 256GB. This isn't a perfect measurement – but in the cloud, you can't price components separately.



Why so slow & expensive?

Storage isn't local: it's a shared service.

Storage is a highly reliable building block:

- Individual disks can fail w/o customers knowing
- Servers hosting those disks can also fail
- Customers can't notice lost writes
- Writes must be done on multiple servers, and in most cases, multiple data centers



Deck 3 p21



Cloud storage prices include:

Multiple drives

In multiple servers

In multiple data centers

Power & networking

Custom software to make it all work

People to build & manage it all



Deck 3 p22



How does this affect SQL Server?

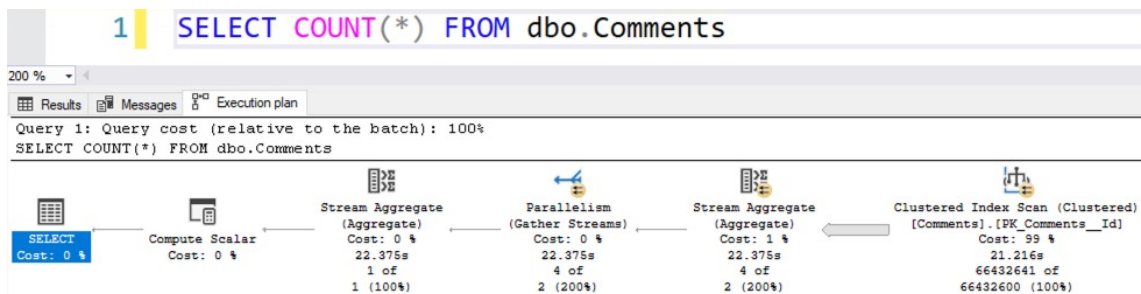
	2000s	2010s	2020s	2020s Cloud
Cost for 128GB RAM	\$12,800	\$1,280	\$500	\$11,700/yr* (E16bds_v5)
Typical drive IOPs	150	150,000	1,500,000	5,000 (Azure P30 1TB)
Typical drive throughput	100 MB/sec	1,000 MB/sec	7,000 MB/sec	200 MB/sec (Azure P30 1TB)

Deck 3 p23



Let's take this query.

My Comments table is just 21GB in size:



Deck 3 p24



How does this affect SQL Server?

	2000s	2010s	2020s	2020s Cloud
Cost for 128GB RAM	\$12,800	\$1,280	\$500	\$11,700/yr* (E16bds_v5)
Typical drive IOPs	150	150,000	1,500,000	5,000 (Azure P30 1TB)
Typical drive throughput	100 MB/sec	1,000 MB/sec	7,000 MB/sec	200 MB/sec (Azure P30 1TB)
Time to read 21GB:	210 seconds	21 seconds	3 seconds	105 seconds

Deck 3 p25

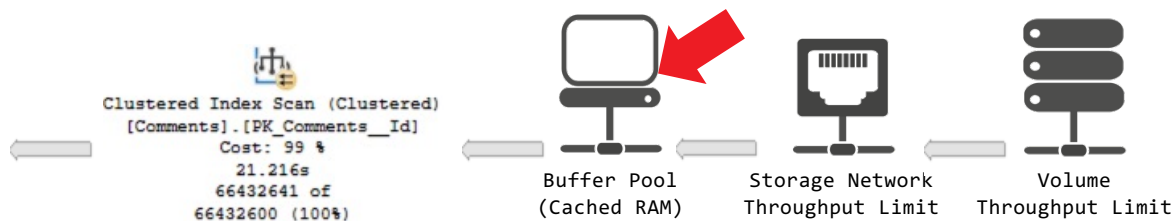


Doesn't memory fix this problem?

Deck 3 p27



If you can cache all data in RAM



Then storage throughput only matters:

- On startup
- For backups
- For inserts/updates/deletes (to the log file)

Deck 3 p28



But if you can't cache it ALL...

Then caching some of a table doesn't usually work.

To keep it simple, say our query is:

```
SELECT *  
FROM dbo.Comments  
WHERE UserId = 26837      /* Brent */  
ORDER BY Id;              /* Clustered primary key */
```

Deck 3 p29



The query plan without an index

```
3 SELECT *  
4 FROM dbo.Comments  
5 WHERE UserId = 26837      /* Brent */  
6 ORDER BY Id;              /* Clustered primary key */
```

200 %

Messages Execution plan

Query 1: Query cost (relative to the batch): 100%

SELECT * FROM dbo.Comments WHERE UserId = 26837 /* Brent */ ORDER BY Id

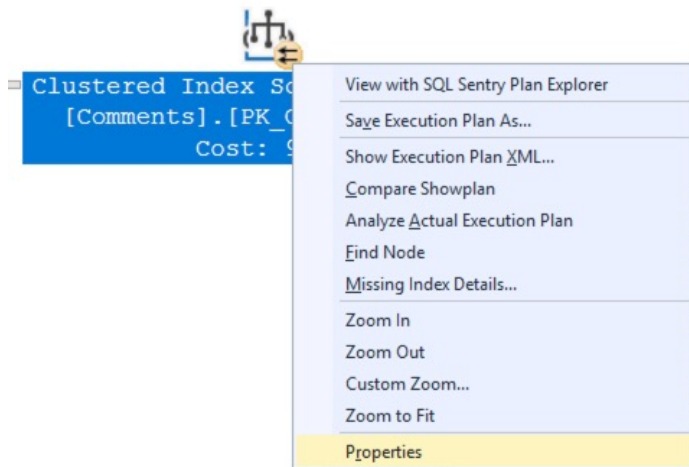
Missing Index (Impact 99.9966): CREATE NONCLUSTERED INDEX [<Name of Missing Index, sysname,>] ON [dbo].[Comments] ([PK_Comments_Id])



Deck 3 p30



Right-click on the CX scan,
click Properties

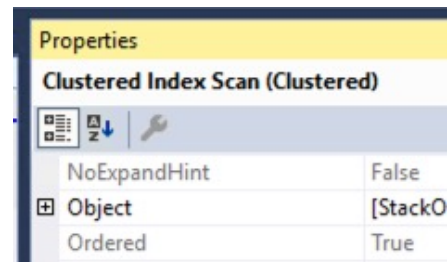


Deck 3 p31



Ordered = True

SQL Server is scanning
the clustered index
in order, from start to end



So if you can't cache the entire table,
something terrible happens

Deck 3 p32



Comments clustered index

The first time the query runs,
SQL Server asks for the first few pages...



Deck 3 p33



Say we have 8GB memory

We may only be able to cache 4GB of the table

The part we're caching is like a sliding window

As our query progresses, the window moves...



Deck 3 p34



Say we have 8GB memory

We may only be able to cache 4GB of the table

The part we're caching is like a sliding window

As our query progresses, the window moves...



Deck 3 p35



Say we have 8GB memory

We may only be able to cache 4GB of the table

The part we're caching is like a sliding window

As our query progresses, the window moves...



Deck 3 p36



Say we have 8GB memory

We may only be able to cache 4GB of the table

The part we're caching is like a sliding window

As our query progresses, the window moves...



Deck 3 p37



When our query finishes

We have some of the Comments table cached...

But...

If the query runs again...



Deck 3 p38



When the next query starts

SQL Server wants the first few pages

They're not cached, so we have to fetch 'em from disk

Result: constantly hammering storage



Deck 3 p39



Does adding memory help?

Say we upgrade the server to 24GB

And say 18GB of that is available for the buffer pool

When the query finishes, it looks like this:



Deck 3 p40



We're still screwed.

Adding some memory helps when:

- Tables are small
- Readaheads are the problem
- Query workspace is the problem

Caching the entire working data set:
THAT's where the gains are.

More details: <https://www.brentozar.com/archive/2022/07/why-adding-some-memory-doesnt-fix-all-caching-problems/>

Deck 3 p41



How did we cope in the 2000s?

We couldn't cache the entire data set back then:
memory was too expensive.

So what did we do?

	2000s	2010s	2020s	2020s Cloud
Cost for 128GB RAM	\$12,800	\$1,280	\$500	\$11,700/yr* (E16bds_v5)
Typical drive IOPs	150	150,000	1,500,000	5,000 (Azure P30 1TB)
Typical drive throughput	100 MB/sec	1,000 MB/sec	7,000 MB/sec	200 MB/sec (Azure P30 1TB)

Deck 3 p42



How Alex the Architect coped



Paid close attention to how servers connected to the storage network

Put a lot of work into picking hardware

Benchmarked HW before going live

Read tech news closely, eager for better options, quickly adopted new stuff like Fusion-IO drives

Deck 3 p43



How Billie the Small Biz coped



Tried storage pools: bought more drives, spread the load across 'em

Teamed 1Gb Ethernet cards together trying to get more throughput

Willing to do one-off server builds with unusual configurations (low CPU, high RAM) to offset problems

Deck 3 p44





How Elliot's Enterprise coped

Brought in consultants, HW vendors, MS

Adopted reference configurations like the MS Fast Track Architectures for data warehouses, virtualization

With those standards in place, performance just was what it was: enterprises don't react fast or react by buying bleeding edge hardware

Deck 3 p45



What they all
had in common:

RAID



Redundant Array of Independent Drives

Group drives together for:

- Protection: because hard drives failed often, keep multiple copies of data
- Speed: because each one was pretty slow, spread the same data out across lots of drives

Deck 3 p47



Redundant Array of Independent Drives

Group drives together for:

- Protection: because hard drives failed often, keep multiple copies of data

Protection matters less in the cloud
not because individual drives are more reliable,
but because failures are mostly invisible to you anyway.
The cloud provider builds in redundancy for that.

Deck 3 p48



Redundant Array of Independent Drives

Group drives together for:

But this still matters a lot in the 2020s:

- **Speed:** because each one was pretty slow, spread the same data out across lots of drives

Because cloud storage is pretty crappy.

Deck 3 p49

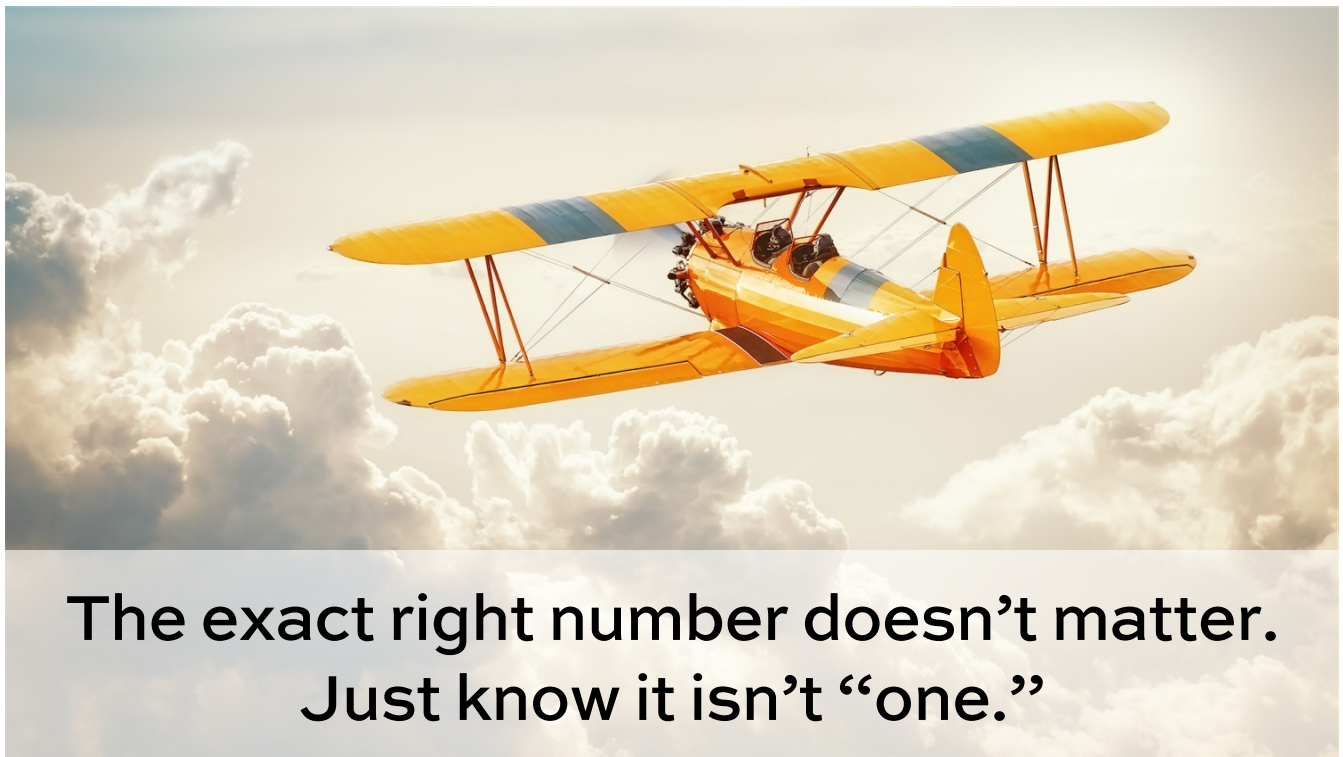


How many drives would you need?

	IOPs (peak burst)	MB/sec	# of drives it would take to match a single laptop SSD
Samsung 990 Pro laptop drive	1,500,000	7,450	-
Azure Premium P30 1TB	30,000	1,000	7-50
Azure Premium SSD v2 Preview	80,000	1,200	6-19
Azure Ultra	160,000	4,000	2-9
AWS GP3 (like Azure Premium)	16,000	1,000	7-94
AWS Provisioned IOPs	64,000	1,000	8-24

Deck 3 p50





The exact right number doesn't matter.
Just know it isn't "one."

The old way to use many drives

Present each one as a separate volume to Windows:

- L:\
- M:\
- N:\
- O:\ ... etc

Break each database up into multiple data files

Put each file on its own storage volume



Problems with that approach

Need to add a new volume for more performance?

- In every database: add a new data file
- Rebuild indexes to spread the data around
- That's a slow, logged activity
- It's really hard to balance things out evenly

Need to subtract a volume because load died off?
Forget it. Quit your job. Move to another city.

Deck 3 p53



The new way: let the OS do it

Add another physical volume

Windows admins configure Storage Spaces:
RAID, but managed in software

Windows can even mirror writes across VMs
(but I'm not a fan: remember, we're doing it for speed)

SQL Server just sees one growing logical volume

No need to rebalance data across files

Deck 3 p54



“But what if a drive fails?”

You need alerting just like you’d need on bare metal.

Just like we did back in the 2000s.

If your drive pool has redundancy,
Windows admins need to add another volume in.

If it doesn’t have redundancy (striping),
SQL Server will go down. That’s bad.

Just like RAID worked back in the 2000s.

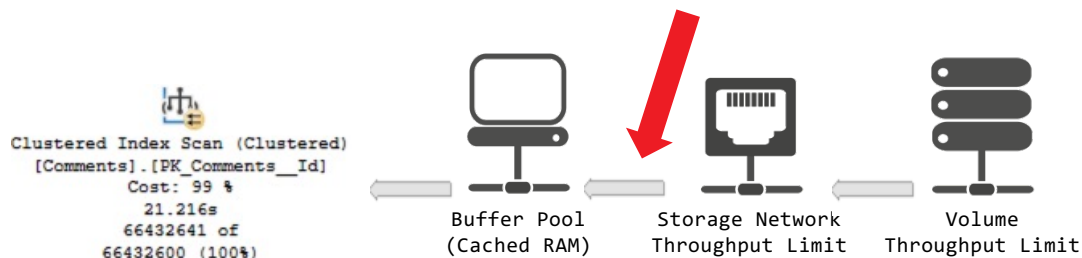
Deck 3 p55



“How fast is fast enough?”

We’ll talk about benchmark testing
in the next module.

But before we get to that,
we need to talk about the other bottleneck.



Deck 3 p56



Picking the right VM type



Back in the 2000s

SQL Server: more, faster drives Web servers: boot drive mirror



That's true in the cloud, too.

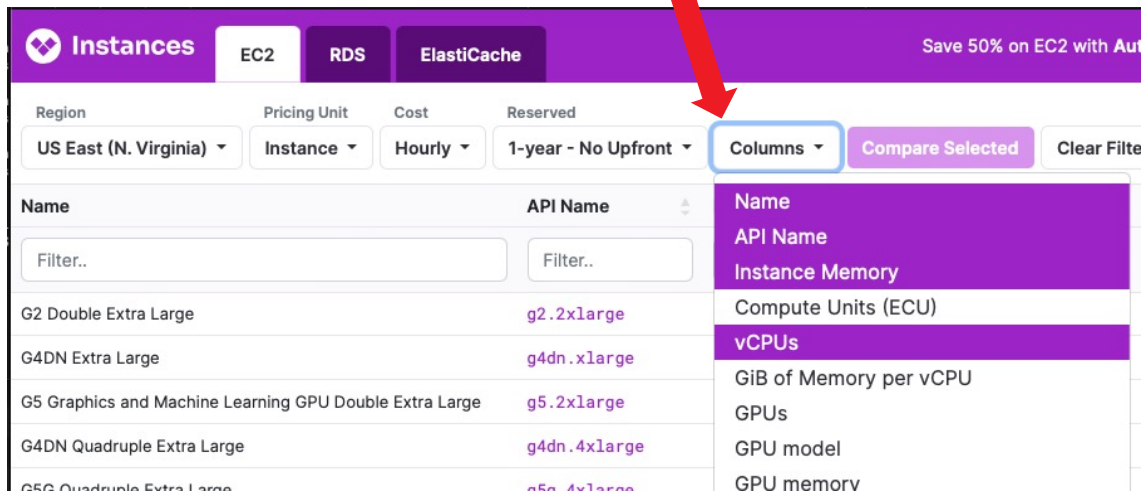
Different VM types have different:

- # of CPU cores available
- Makes/models of CPU, different speeds
- Amounts of memory available per CPU core
- Amounts of local ephemeral solid state storage
- **Sizes of connectivity to storage**

To see it: <https://ec2instances.info> (awesome)
<https://azure-instances.info> (not nearly as good)



Click the Columns dropdown,
pick the columns you want



What I like to show

VM columns:

API Name: i3.4xlarge

Instance memory (RAM)

vCPUs (cores)

GiB Memory per vCPU

Clock Speed

Cost per year

AWS storage columns:

EBS Optimized:
Max Bandwidth

EBS Optimized:
Max Throughput

EBS Optimized:
Max IOPs

Azure storage columns:
Premium Storage ones



Let's go through a sizing exercise.

Alex the Architect's current SQL Server:

- 8 cores of Standard Edition licensing
- 64GB RAM
- Database is about 250GB (too big to fit in memory)

What hardware do we get in the cloud?



Deck 3 p63



Instances						
EC2 RDS ElastiCache						
Save 50% on EC2 with Autopilot, automated Re						
Region	Pricing Unit	Cost	Reserved			
US East (N. Virginia)	Instance	Annually	1-year - No Upfront	Columns	Compare Selected	Clear Filters
API Name	Instance Memory	vCPUs	GiB of M	peed(GHz)	EBS Optimized: Max Bandwidth	
Filter..	Min Mem: 0	8	Min Mem/vCPU: 0	Filter..	Filter..	
m6g.2xlarge	32.0 GiB	8 vCPUs	4.00 GiB/vCPU	unknown	4750.0 Mbps	
m6gd.2xlarge	32.0 GiB	8 vCPUs	4.00 GiB/vCPU	2.5 GHz	4750.0 Mbps	
m6i.2xlarge	32.0 GiB	8 vCPUs	4.00 GiB/vCPU	3.5 GHz	10000.0 Mbps	
m6id.2xlarge	32.0 GiB	8 vCPUs	4.00 GiB/vCPU	3.5 GHz	10000.0 Mbps	
p3.2xlarge	61.0 GiB	8 vCPUs	7.62 GiB/vCPU	2.3 GHz	1750.0 Mbps	
r3.2xlarge	61.0 GiB	8 vCPUs	7.62 GiB/vCPU	2.5 GHz	1000.0 Mbps	
r4.2xlarge	61.0 GiB	8 vCPUs	7.62 GiB/vCPU	2.3 GHz	1700.0 Mbps	
r5.2xlarge	64.0 GiB	8 vCPUs	8.00 GiB/vCPU	3.1 GHz	4750.0 Mbps	
r5a.2xlarge	64.0 GiB	8 vCPUs	8.00 GiB/vCPU	2.5 GHz	2880.0 Mbps	
r5ad.2xlarge	64.0 GiB	8 vCPUs	8.00 GiB/vCPU	2.5 GHz	2880.0 Mbps	
r5b.2xlarge	64.0 GiB	8 vCPUs	8.00 GiB/vCPU	3.1 GHz	10000.0 Mbps	

1. Filter for 8 cores

Then pick out the good deals.

Sure, as you set your thresholds, the options thin out.

Instances							
EC2		RDS	ElastiCache	Save 50% on EC2 with Autopilot, automated Reserved Instance purchasing.			
Region	Pricing Unit	Cost	Reserved				
US East (N. Virginia)	Instance	Hourly	1-year - No Upfront	Columns	Compare Selected	Clear Filters	
API Name	Instance Memory	vCPUs	GIB of Memory per vCPU	Clock Speed(GHz)	EBS Optimized: Max Bandwidth	Windows Reserved cost	
Filter..	64	8	Min Mem/vCPU: 0	3.	Filter..	Filter..	
x2iedn.2xlarge	256.0 GiB	8 vCPUs	32.00 GiB/vCPU	3.5 GHz	20000.0 Mbps	\$12210.213600 annually	
d3.2xlarge	64.0 GiB	8 vCPUs	8.00 GiB/vCPU	3.1 GHz	2800.0 Mbps	\$8733.720000 annually	
i3en.2xlarge	64.0 GiB	8 vCPUs	8.00 GiB/vCPU	3.1 GHz	4750.0 Mbps	\$8619.840000 annually	
r5.2xlarge	64.0 GiB	8 vCPUs	8.00 GiB/vCPU	3.1 GHz	4750.0 Mbps	\$6009.360000 annually	
r5d.2xlarge	64.0 GiB	8 vCPUs	8.00 GiB/vCPU	3.1 GHz	4750.0 Mbps	\$6403.560000 annually	
i4i.2xlarge	64.0 GiB	8 vCPUs	8.00 GiB/vCPU	3.5 GHz	10000.0 Mbps	\$7121.880000 annually	
r6i.2xlarge	64.0 GiB	8 vCPUs	8.00 GiB/vCPU	3.5 GHz	10000.0 Mbps	\$6144.264000 annually	
r5dn.2xlarge	64.0 GiB	8 vCPUs	8.00 GiB/vCPU	3.1 GHz	4750.0 Mbps	\$6911.640000 annually	
r6id.2xlarge	64.0 GiB	8 vCPUs	8.00 GiB/vCPU	3.5 GHz	10000.0 Mbps	\$6553.356000 annually	
r5b.2xlarge	64.0 GiB	8 vCPUs	8.00 GiB/vCPU	3.1 GHz	10000.0 Mbps	\$6512.884800 annually	

<https://aws.amazon.com/ec2/instance-types/r6i/>

aws

Contact Us Support English My Account Sign In to the Console

Products Solutions Pricing Documentation Learn Partner Network AWS Marketplace Customer Enablement Events Explore More

Amazon EC2 Overview Features Pricing Instance Types FAQs Getting Started Resources

Amazon EC2 R6i Instances

Run memory-optimized instances powered by 3rd Generation Intel Xeon Scalable processors

Get started with Amazon EC2 R6i Instances

Innovate Every Application Edition | Learn how to upgrade your application at our free online conference on 9/28

Amazon EC2 R6i instances are powered by 3rd Generation Intel Xeon Scalable processors and deliver up to 15% better price performance compared to R5 instances. R6i instances feature an 8:1 ratio of memory to vCPU, just like R5 instances, and support up to 128 vCPUs per instance, which is 33% more than R5 instances. These instances are SAP-Certified and are an ideal fit for memory-intensive workloads, such as SQL and NoSQL databases; distributed web scale in-memory caches, such as Memcached and Redis; in-memory databases, such as SAP HANA; and real-time big data analytics, such as Apache Hadoop and Apache Spark clusters. R6id instances come with local NVMe-based SSD block-level storage for applications that need high-speed, low-latency local storage. Compared to previous generation R5d instances, R6id instances offer 58% higher TB storage per vCPU and 34% lower cost per TB.



Amazon EC2 R6id instances now available

R6id instances are powered by 3rd Generation Intel Xeon Scalable processors with an all-core turbo frequency of 3.5 GHz, up to 7.6 TB of local NVMe-based SSD block-level storage, and up to 15% better price performance than previous generation instances. R6id are ideal for a wide range of memory-intensive workloads including those that need access to high-speed, low-latency local storage.

<https://aws.amazon.com/ec2/instance-types/r6i/>

Amazon EC2 R6i instances are powered by 3rd Generation Intel Xeon Scalable processors and deliver up to 15% better price performance compared to R5 instances. R6i instances feature an 8:1 ratio of memory to vCPU, just like R5 instances, and support up to 128 vCPUs per instance, which is 33% more than R5 instances. These instances are SAP-Certified and are an ideal fit for memory-intensive workloads, such as SQL and NoSQL databases; distributed web scale in-memory caches, such as Memcached and Redis; in-memory databases, such as SAP HANA; and real-time big data analytics, such as Apache Hadoop and Apache Spark clusters. R6id instances come with local NVMe-based SSD block-level storage for applications that need high-speed, low-latency local storage. Compared to previous generation R5d instances, R6id instances offer 58% higher TB storage per vCPU and 34% lower cost per TB.



Amazon EC2 R6id instances now available

R6id instances are powered by 3rd Generation Intel Xeon Scalable processors with an all-core turbo frequency of 3.5 GHz, up to 7.6 TB of local NVMe-based SSD block-level storage, and up to 15% better price performance than previous generation instances. R6id are ideal for a wide range of memory-intensive workloads including those that need access to high-speed, low-latency local storage.

Came out November 2021.

Deck 3 p71



Gradual rollouts by region

When the R6i dropped, wasn't available everywhere:

Availability and Pricing

EC2 R6i instances are available today in four [AWS Regions](#): US East (N. Virginia), US West (Oregon), US East (Ohio), Europe (Ireland). As usual with EC2, you pay for what you use. For more information, see the [EC2 pricing page](#).

Then one month later:

Posted On: Dec 13, 2021

Starting today, Amazon [EC2 R6i instances](#) are available in additional AWS Regions US West (N. California), Asia Pacific (Mumbai), Asia Pacific (Seoul), Asia Pacific (Singapore), Asia Pacific (Sydney), Asia Pacific (Tokyo), Europe (London), and Europe (Paris). Designed for

Deck 3 p72



Why everyone doesn't use the same instance types

Not all instance types are available everywhere

Not everyone can take outages to move to new types

Not everyone needs fast CPUs, lots of RAM, fast storage throughput

Old VM types can be less expensive
(but pricing is all over the map, changes frequently)

Deck 3 p73



What we're going to need to know

How much memory do we have today?

How much storage throughput do we have today?
(Because if data isn't cached, this is the bottleneck.)

How many cloud drives would it take to get that same throughput?

What VM type do we need to handle that much throughput?

Could we compensate by throwing memory in the box?

Deck 3 p74



For example, say...

Our current server: 6,000 MB/sec read throughput.

1 EBS Provisioned IOPs volume max: 1,000 MB/sec

$6,000 / 1,000 =$ we need at least 6 EBS volumes.

And our EC2 virtual machine needs to be able to handle 6,000 MB/sec – many can't.

API Name	Instance Memory	vCPUs	GiB of Memory per vCPU	Clock Speed(GHz)	EBS Optimized: Max Bandwidth	Windows Reserved cost
Filter..	64	8	Min Mem/vCPU: 0	3.	Filter..	Filter..
x2iedn.2xlarge	256.0 GiB	8 vCPUs	32.00 GiB/vCPU	3.5 GHz	20000.0 Mbps	\$12210.213600 annually
d3.2xlarge	64.0 GiB	8 vCPUs	8.00 GiB/vCPU	3.1 GHz	2800.0 Mbps	\$8733.720000 annually
i3en.2xlarge	64.0 GiB	8 vCPUs	8.00 GiB/vCPU	3.1 GHz	4750.0 Mbps	\$8619.840000 annually

Nope

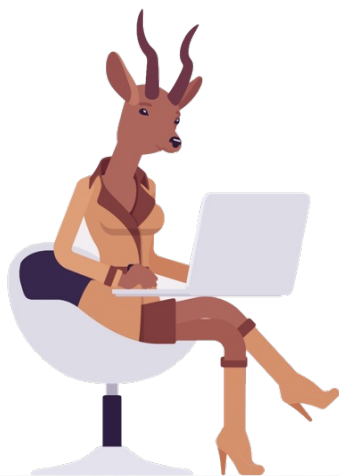
We don't have all the answers yet.

Alex the Architect's current SQL Server:

- 8 cores of Standard licensing
- 64GB RAM
- Database is about 250GB (too big to fit in memory)

But we haven't measured Alex's storage throughput – or if it's a problem.

We'll dig into that after backups.



Recap

What we discussed

Storage hardware is hidden in plan read operators

From 2000s to 2020s,
RAM & storage improved 20X-1000X

Cloud hardware regressed back to the 2000s

To handle it, we have to use 2000s-era techniques:

- Get pools of drives for speed, not 1 for capacity
- Pick your VM instance type based on throughput

